

How Many Centimetres Does the Depth Source Cost?

A Single-Variable Pipeline for Comparing Laser, Mobile LiDAR, and Monocular Depth in Room Reconstruction

Author Name
Affiliation
email@example.com

Abstract—Reconstructing a room in 3D from a phone currently depends on a LiDAR sensor that only a minority of devices carry. Monocular depth models have improved dramatically on single images, but there is no published number for how many centimetres a room drifts when such a model replaces LiDAR inside an otherwise unchanged reconstruction pipeline. This system-development paper builds a classical pipeline—per-frame depth → scale alignment → TSDF fusion → mesh—in which *the depth source and the scale-recovery strategy are interfaces switched from a config file*, and measures mesh geometry against a laser-scanner reference on six real rooms from ARKitScenes (2,352 frames) under identical conditions. Results: the pipeline itself loses 2.1 ± 0.5 cm (Chamfer, 4 cm voxels); iPad Pro LiDAR adds only 0.98 ± 0.25 cm (F@5 cm 0.93); Depth Anything V2 with the correct scale on every frame loses 5.3 ± 1.3 cm (F@5 cm 0.79), 2.29 cm behind LiDAR. With scale calibrated only once per room the error jumps to 22.9 ± 2.1 cm (F@5 cm 0.23), and an uncalibrated “metric” model gives 54.3 cm. Per-frame analysis shows the cause: the model’s scale swings by several times with the content of each view, not with time. Fitting scale per frame on only ~ 200 sparse metric points—the kind a phone’s visual-inertial tracker provides for free—recovers 6.0 cm (F@5 cm 0.74), essentially the oracle. The alternative is to give the model the missing scale by training: fine-tuning the same metric checkpoint on ARKitScenes (frozen encoder, DPT head only) brings the uncalibrated row to 14.6 cm when the training labels come from the laser scanner and 16.6 cm when they come from an iPad’s own LiDAR instead (AbsRel 0.136 vs 0.136)—so the supervision a \$20k scanner provides can be collected with a consumer depth sensor, and devices without one inherit it. Model size (Large→Small) costs just 2.8 cm for a $7\times$ speed-up, and frame rate governs coverage rather than accuracy (≥ 0.8 fps is needed). The product conclusion is that the gap between devices with and without LiDAR is a per-frame scale problem, solvable with metric data from VIO rather than with a larger model. Code, every experiment config, and every run’s metrics are released.

Index Terms—monocular depth estimation, TSDF fusion, 3D room reconstruction, LiDAR, scale alignment, ARKitScenes

I. INTRODUCTION AND BUSINESS MOTIVATION

A. Problem

Scanning a room into a 3D model is already a consumer feature on iPhone Pro and iPad Pro through Apple RoomPlan and apps such as Polycam, all of which rely on the *LiDAR*

sensor for directly metric depth. Devices with LiDAR are, however, a minority of the market: non-Pro iPhones and almost every Android phone have none. The alternative for these devices is *monocular depth estimation*—networks that predict depth from a single RGB image—which advanced rapidly between 2020 and 2024 (MiDaS, Depth Anything) to the point of producing visually convincing results on single images.

The question a product team must answer before investing is: **if LiDAR is replaced by a monocular model while the rest of the system stays the same, by how many centimetres is the reconstructed room wrong, and which use cases does that still serve** (floor-area estimation, furniture placement, renovation measurement)? Monocular-depth research reports per-image metrics (AbsRel, δ_1) on benchmarks, which do not translate into “how crooked is the wall.” End-to-end reconstruction systems (Atlas, NeuralRecon, SimpleRecon) design their whole pipeline around their own model, so the *depth source* cannot be compared in isolation.

B. Approach

This is a *system-development* study. We build a classical room-reconstruction pipeline (per-frame depth → scale alignment → TSDF voxel fusion → mesh) in which **the depth source is the single swappable variable**, and measure the mesh geometry against a laser-scanner reference on real rooms (ARKitScenes) with three sources through one and the same pipeline:

Isolating *ScaleAligner* as a separate stage lets us separate “wrong shape” from “wrong scale” in the monocular rows: *oracle_frame* (fit per frame against GT) is the model’s ceiling; *per_scene* (one calibration) is what a one-off calibration achieves; *sparse_points* (fit per frame on a few hundred metric points) is what a deployed system with a VIO tracker can do.

C. Main results (6 scenes, 4 cm voxels)

- The pipeline itself loses 2.1 ± 0.5 cm (Chamfer) when fed laser depth; iPad LiDAR loses 3.0 ± 0.6 cm—a 1 cm gap

Depth source	Stands for
Faro laser (highres_depth)	upper bound of the pipeline: its own loss with perfect depth
ARKit LiDAR (lowres_depth, 256×192)	what iPhone Pro does today
Monocular (Depth Anything V2, MiDaS) $\times$ scale strategy	what a device without LiDAR can do

that is the same in every room type.

- Monocular depth with the correct scale on every frame (oracle) loses 5.3 ± 1.3 cm: 2.29 cm behind LiDAR, i.e. “Android is about 2 cm behind iPhone Pro *if* the scale problem is solved.”
- Monocular depth calibrated once per room loses 22.9 ± 2.1 cm, uniformly across rooms; a “metric” model without calibration loses 54.3 ± 10.6 cm. **The real problem of monocular depth is not shape but a scale that swings per view** (§VI).
- Fitting scale per frame on ~ 200 sparse metric points closes that gap to 6.0 cm (F@5 cm 0.74)—within 0.6 cm of the oracle (§V).
- Fine-tuning the metric model on ARKitScenes fixes the uncalibrated row without any alignment at run time: 14.6 cm with a laser teacher, 16.6 cm with a consumer-LiDAR teacher ($1.13\times$), from 54.3 cm pretrained (§V-F).
- A $13\times$ smaller model (DA-v2 Small) costs 2.8 cm but runs $7\times$ faster; MiDaS small is a further $4.5\times$ faster with $2.3\times$ the error of Large. Model size matters $6\times$ less than scale.
- Frame count governs coverage, not accuracy: ≥ 0.8 fps is needed to capture the whole room.

D. Contributions

- 1) An open-source pipeline in which DepthSource and ScaleAligner are interfaces switched from config without touching the orchestrator, so every experiment table is “one loop, one variable” and the same code serves as a product backend (§III).
- 2) “Centimetres on the wall” for Faro / LiDAR / monocular depth through one pipeline on six real rooms with a laser-scanner reference (§IV-V).
- 3) An analysis showing that the oracle $\rightarrow$ per-scene gap comes from per-view scale, not temporal drift, that ~ 200 sparse metric points per frame recover the oracle, and what that implies for system design (§VI).
- 4) A controlled answer to “can a phone’s own depth sensor replace the laser scanner as the *teacher*?”: the same fine-tuning recipe, the same scenes, only the training label changes—Faro 14.6 cm vs. ARKit LiDAR 16.6 cm on held-out rooms, with identical AbsRel. Checkpoints and the scene split are released (§V-F).
- 5) Per-scan cost (time, frames, model size, LiDAR confidence weighting) for business decisions (§VII).

a) What this paper does not claim: It does not claim to beat Atlas / NeuralRecon / SimpleRecon—they are not compared, and this pipeline is not designed to compete. It does not claim that “not needing LiDAR” is a feature—the data say the opposite: without LiDAR some per-frame metric

information is still required, or the model must be taught the scale beforehand (§V-F), which is still 11.2 cm short of having metric anchors at run time. Only the metric head of one model is fine-tuned (frozen encoder, ARKitScenes); every other model is used pretrained.

II. RELATED WORK

A. Monocular depth estimation

Monocular depth models fall into two families according to what their output promises.

a) Relative / affine-invariant: MiDaS [1] introduced training across many datasets with a loss that ignores scale and shift, giving strong cross-domain generalisation at the price of an output that is *disparity up to an affine transform*: two constants (s, t) per image must be known to obtain metres. Depth Anything [2] and Depth Anything V2 [3] extend the idea with large unlabelled corpora and synthetic teachers, giving much sharper edges and detail while remaining affine-invariant in disparity. We use DA-v2 (Large / Small) and MiDaS small as representatives of this family and **treat (s, t) as an experimental variable** (the ScaleAligner) rather than hiding it in the model.

b) Metric: ZoeDepth [4] and the metric variants of DA-v2 (fine-tuned on Hypersim / Virtual KITTI) aim to output metres directly. Metric3D [5] identifies the camera focal length as the main ambiguity and resolves it by normalising images to a canonical camera. These works report per-image AbsRel/δ_1 on NYUv2/KITTI; our results (§VI-B) show that on real room video the per-frame scale of a metric model still swings from -60% to $+80\%$ within one scan—invisible to per-image metrics, immediately visible once frames are fused into one volume.

c) Conditioning on the camera, or on a teacher: Two families answer the scale ambiguity differently. One conditions the network on camera intrinsics—Metric3D [5], UniDepth [6], and Depth Pro [7], which estimates the focal length itself—so a metre prediction is tied to a known field of view. The other adapts a pretrained model to the target domain with depth labels, which is what the metric DA-v2 variants do on Hypersim. We take the second route but change where the labels come from: instead of a laser scanner or a synthetic renderer, the teacher is the depth sensor already shipping in consumer phones. This puts a price on the teacher—the question a product team actually faces—rather than on the architecture, and it is measured in mesh centimetres on held-out rooms rather than in per-image AbsRel (§V-F). The two families are complementary: an intrinsics-conditioned model could be fine-tuned with the same LiDAR labels.

d) *Mobile LiDAR depth*: ARKit provides 256×192 depth fusing LiDAR with imagery; ARKitScenes [8] is the first dataset to release both this depth and Faro laser-scanner depth for the same rooms, making a direct “mobile LiDAR vs. laser” comparison possible. We use both as two rows of the same table.

B. Multi-view 3D reconstruction from video

a) *Classical depth fusion*: Volumetric TSDF fusion [9] and KinectFusion [10] combine many depth frames through a truncated signed distance per voxel and extract the surface with marching cubes. The property we need is that **fusion neither knows nor cares where depth comes from**—as long as it is metric and posed—so it is a fair “fixed part” for every source. We use the Open3D implementation [11].

b) *End-to-end learned reconstruction*: Atlas [12] regresses a TSDF directly from multi-image features, Neural-Recon [13] does so incrementally in real time, and SimpleRecon [14] shows that good multi-view depth plus classical fusion is competitive. These are strong ScanNet baselines, but all learn depth *from multiple images*; they do not answer “what room do you get from a per-image monocular model plus the device’s VIO pose,” which is the situation of an app on a device without LiDAR. We therefore **do not compare against this group** and make no claim of superiority.

c) *Scale in monocular SLAM / reconstruction*: Using monocular depth inside SLAM requires per-frame scale recovery, e.g. from sparse tracked points (CNN-SLAM [15] and successors), or joint optimisation of scale/shift with pose (MonoSDF [16] uses monocular priors in a neural surface). Our `sparse_points` aligner follows this line; we evaluate it next to `oracle_frame` and `per_scene` so that the “price of not knowing scale” is measured at three levels of available metric information.

C. Room-scanning products

Apple RoomPlan [17] produces a floor plan and furniture boxes from LiDAR on iPhone/iPad Pro; Polycam and similar apps produce meshes or photogrammetry. All require LiDAR for room mode or fall back to slow offline photogrammetry with many photos. No public report states how many centimetres these products deviate from a laser scanner; this paper provides that number at least for our own pipeline.

D. Position of this work

III. SYSTEM DESIGN

A. Design principle: one pipeline, one variable

The research question (“change only the depth source—how much changes?”) imposes three architectural requirements:

- 1) **Every depth source must pass through the same code after the point where it is produced**—no special branch for mono or LiDAR in fusion or evaluation.
- 2) **Whatever can be swapped is swapped from config**, not by editing code: one result table = one YAML file.

- 3) **The paper’s code is the product’s code**: the same system must accept ARCore depth or a smaller model on the day an MVP ships, without a rewrite.

Requirement 1 leads to the `DepthSource` interface; requirement 2 to a string→class registry and base-plus-override configs; requirement 3 to a single monolithic package with no service, database or experiment tracker—appropriate for a one-person, ten-week project.

B. Overview

Fig. 1 shows the data flow. Per frame there are three typed hand-over points:

Invariants enforced in code, not just documented: (i) values are metres after the aligner; (ii) `pose_c2w` is camera-to-world and is inverted in exactly one place (`TSDFFusion.integrate`); (iii) intrinsics are bound to a resolution—resizing requires `Intrinsics.scaled()`; (iv) `is_metric=True` $\Rightarrow$ `aligner=identity` and `is_metric=False` $\Rightarrow$ `aligner!=identity`—the pipeline fails at construction if the pair is wrong.

C. DepthSource: depth as a strategy

```
class DepthSource(ABC):
    name: str
    is_metric: bool
    def get_depth(self, frame) -> np.ndarray
        # (H,W) float32; 0/NaN = invalid
```

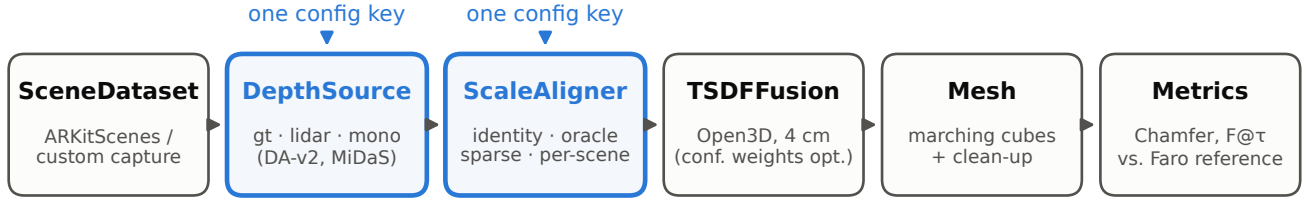
It takes the whole `Frame` rather than an RGB image because each source needs different inputs: `gt` reads `frame.gt_depth`, `lidar` reads `frame.extra["lidar_depth"]` (plus confidence), `mono` reads `frame.rgb`. One signature covers all cases without edits when `arcore` is added. `MonocularDepth` does three things that belong neither to the model nor to the aligner: it rotates the image upright according to `sky_direction`, converts disparity to $1/x$ so that “larger = farther,” and masks invalid values. Models (`DepthModel: DA-v2, MiDaS`) form a second registry under `mono`, so Exp. 4 (model size) is just `depth.model=...`

D. ScaleAligner: scale as a separate stage

A relative model gives depth up to an affine transform. How (s, t) is obtained is a **system decision** that should be measured separately from the model, so it is its own stage: The gap `oracle_frame` $\rightarrow$ `per_scene` is “the price of not knowing scale”—an experimental result, not noise; `sparse_points` measures how much of that price a phone’s tracker can pay back.

a) *Fitting space*: Affine-invariant models are affine in *disparity*, not depth; fitting $gt \approx s \cdot pred + t$ in depth space is the wrong model. We therefore fit $1/gt \approx s \cdot (1/pred) + t$ (least squares with three rounds of trimming the worst 20% of residuals) and invert back. On scene 42444474 this change alone moved `mono+oracle` from 15.2 cm to 5.4 cm—an example of why the aligner must be testable in isolation.

	Trains	Compares depth sources	Reference	Metrics
Monocular depth papers	✓	× (one model)	per-image depth	AbsRel, δ
Atlas / NeuralRecon / SimpleRecon	✓	× (multi-view only)	ScanNet mesh	Chamfer, F-score
This work	head only (LiDAR teacher)	✓ laser / LiDAR / mono × aligner × teacher	Faro laser (ARKitScenes)	Chamfer, F@ τ , per-frame AbsRel/ δ , time



frames: RGB + pose (VIO) + optional sensor depth / confidence / sparse points

Fig. 1. Pipeline components. The two highlighted boxes are the only things that change between rows of a result table; everything to their right is fixed (Open3D ScalableTSDFVolume, marching cubes, cluster filtering). Frame stride (Exp. 3) is a dataset parameter; voxel size (Exp. 2) is a fusion parameter.

Point	Type	Owner
Frame{rgb, pose_c2w, gt_depth?, extra}	from SceneDataset	dataset mapping
pred: (H,W) float32, relative or metric	DepthSource.get_depth(frame)	source declares is_metric
depth_m: (H,W) float32 metres, 0 = invalid	ScaleAligner.align(pred, frame)	invariant: metric after the aligner

Aligner	Fits (s, t)	Meaning
identity	—	GT / LiDAR / metric models
oracle_frame	per frame, against GT depth	model ceiling (not deployable)
per_scene	once, on N frames spread over the scan, then frozen	one-off calibration
sparse_points	per frame, against ~ 200 sparse metric points	the MVP path (VIO feature points)

b) Sparse points: ARKitScenes ships no ARKit feature points, so `sparse_points` is evaluated against a proxy: $N=200$ pixels sampled per frame (deterministically) from the LiDAR frame at confidence 2, which is what a tracker hands the pipeline—a few hundred metric depths per frame, nothing dense. The fit is per frame in inverse depth with two rounds of 10% trimming; frames with fewer than 20 usable points reuse the previous frame’s (s, t), as a deployed system would when tracking momentarily loses features. The proxy is optimistic in one respect (LiDAR points are cleaner than triangulated VIO features) and pessimistic in another (VIO points cluster on texture, ours are uniform); we treat the row as an *upper bound of real VIO*.

E. Fixed parts: fusion, post-processing, evaluation

- **Fusion:** Open3D ScalableTSDFVolume, 4 cm voxels, `sdf_trunc=3×voxel`, depth 0.1–5 m; every source is resampled to 256×192 (the LiDAR resolution) before integration for fairness, and only pixels where the reference has depth are integrated (`mask_to_gt`)—no source is penalised for seeing beyond the Faro coverage. An optional per-pixel integer weight from the sensor’s confidence map (`fusion.confidence_weights`) is off for every main result and used only in Exp. 5.
- **Post-processing:** small clusters removed; no decimation.

- **Reference:** ARKitScenes provides no laser mesh; we fuse Faro `highres_depth` from every frame at 1 cm voxels once per scene into `reference_mesh.ply`. The `gt` row at 4 cm therefore measures “pipeline loss,” not zero.
- **3D metrics** (fixed protocol in `metrics_3d.py`): 200k points sampled on both meshes (fixed seed); accuracy = mean `pred→ref` distance, completeness = `ref→pred`, Chamfer = their mean; precision/recall/F-score at 2/5/10 cm (**F@5 cm is the headline**); normal consistency = mean $|n_{pred} \cdot n_{ref}|$.
- **2D metrics** per frame (RMSE, AbsRel, δ_{1-3}) compare depth *after* alignment with GT depth, to tell whether error arises before or after fusion.

F. Config, registry and experiments

`configs/base.yaml` defines every value; `configs/experiments/expN.yaml` = base + a list of runs, each a handful of dotted-key overrides. `roomscan` sweep runs every run × every scene and writes a resolved `config.yaml` plus `metrics.json` per run; `roomscan` report aggregates them into `summary.csv` and one `table.md` per experiment. The tables in §V are copied from those files.

Every kind of extension is one new file plus one registry line, without touching `pipeline.py`: a depth source (AR-Core), a model (a compressed variant), an aligner, a dataset

(a real room captured by the app; `dataio/custom.py` reads the app’s folder format), or an experiment. The rule that guards the design: if adding something requires editing `pipeline.py`, stop and write an architecture decision record. Over the project this happened exactly once (per-pixel confidence weights, two lines), and it is recorded as such.

G. Deliberately not done

GPU TSDF, an experiment tracker, and pose estimation (the dataset’s ARKit VIO poses are used) are out of scope; a minimal web service and viewer exist as a separate package importing the pipeline, and none of these affect the numbers.

IV. EXPERIMENTAL SETUP

A. Dataset: ARKitScenes

We use ARKitScenes [8] because it is the only dataset that provides **all three** things the question needs for the same room: RGB video from a real device (iPad Pro), depth from that device’s LiDAR (`lowres_depth`, 256×192), and depth from a Faro laser scanner registered to the camera poses (`highres_depth`, 1920×1440). The “iPhone Pro today” row and the reference therefore come from one scan, with no cross-sensor alignment of our own.

Two dataset constraints shape the experiments: (1) Faro depth exists only for the frames Apple kept (~ 3 fps), so every source is evaluated **on this same frame set** and Exp. 3 (frame stride) measures coverage more than compute (§V-C); (2) no laser mesh is provided, so we build one (§IV-B).

Scenes (Table I) were shortlisted from metadata without downloading: `highres_depth` must exist, the video must be in the 3DOD split, and the set must span room types (standard / large / glossy-mirror) that are the expected failure cases of monocular depth. Scene 42444474 was used during development, so its numbers are not held-out; we report per-scene tables in §V to show that the conclusions hold on the five scenes the pipeline never saw.

B. Reference mesh

We fuse `highres_depth` from **every** frame with TSDF at 1cm voxels (`sdf_trunc 3cm`) once per scene into `reference_mesh.ply`, and never touch it again. Consequences for reading the tables: the `gt` row runs at the experiment’s voxel size (4cm) and therefore has non-zero Chamfer—the **discretisation loss of the pipeline itself**, a lower bound for every row; the reference covers only what Faro saw, so every source integrates only pixels where Faro has a value (`eval.mask_to_gt`); and the reference uses the same poses as every row, so VIO pose drift is **removed from the comparison by design**—all numbers are the effect of the depth source alone (limitation: §VIII).

C. Pipeline configuration (fixed for every row)

Monocular models (all pretrained): Depth Anything V2 Large (335M parameters) / Small (25M) and DA-v2 Metric-Indoor (Large, fine-tuned on Hypersim); MiDaS v2.1 small (21M) for Exp. 4. Every model receives the upright image;

its output is converted to depth-like ($1/\text{disparity}$) before the aligner.

D. Scale-alignment protocol

Every non-identity aligner fits an affine map **in inverse-depth space**: (s, t) such that $1/gt \approx s \cdot (1/pred) + t$, by trimmed least squares—three rounds, dropping the worst 20% of residuals each round—on pixels where both GT and prediction are valid within 0.1–5m, then inverted back to metres. `per_scene` also uses GT to fit, so it is the **upper bound of one-off calibration**: a real system that calibrates from a ruler or a few VIO points will do no better.

E. Fine-tuning protocol (Exp. 6)

a) *Scenes*: Disjoint at the `visit_id` level from the six test scenes, drawn with seed 0 from the ARKitScenes screening list: 10 *training* scenes that have both a Faro scan and LiDAR (so the two teachers see exactly the same frames), 14 further training scenes with LiDAR only (no laser scan exists for them), and 3 scenes from the official Validation fold, with Faro, used for checkpoint selection. Six test scenes, 3 validation, 24 training; no room, and no other capture of the same room, appears in two of those sets. The split file (`configs/training/splits.yaml`) and the per-scene frame counts are released with the code.

b) *Labels*: R1 uses Faro `highres_depth` (3594 frames over 10 scenes); R2 uses the iPad’s `lowres_depth` with ARKit confidence ≥ 1 on the same 10 scenes (8656 frames—LiDAR exists for frames the laser scan does not cover); R3 uses the same LiDAR labels over all 24 scenes (19896 frames). Depth is resampled to the RGB grid with nearest-neighbour only; pixels beyond 5m or without a label are masked out.

c) *Optimisation*: The DINOv2 encoder is frozen (300M parameters in `eval` mode); only the DPT neck and the metric head train, with SiLog loss ($\beta = 0.15$), AdamW (lr 5×10^{-5} , weight decay 0.01, 200 warm-up steps then cosine), bf16 autocast, batch 2 with gradient accumulation 4, on 518×518 crops of images resized so the short side is 518. R1 and R2 run 6000 steps, R3 12000; validation every 500 steps keeps the checkpoint with the lowest AbsRel on the 3 validation scenes. One run takes about an hour on an RTX 3070 (3.2 GB of VRAM).

d) *At test time*: the fine-tuned models are plain DepthSource rows like any other: RGB in, metres out, `aligner: identity`, no access to depth, poses or intrinsics.

F. Metrics

a) *3D (headline)*: Following `metrics_3d.py` exactly for every row: 200,000 points sampled uniformly on the produced mesh and on the reference (seed 0); accuracy = mean `pred`→`ref` distance (did we build something that is not there?), completeness = `ref`→`pred` (did we miss something?), **Chamfer = their mean**; precision/recall/F-score at $\tau = 2/5/10$ cm—**F@5 cm is the headline** (5 cm $\approx$ the tolerance for

Asset	Resolution / rate	Role
vga_wide RGB	640 × 480, 30 fps	input to monocular models (rotated upright)
lowres_depth + confidence	256 × 192, 60 fps	depth source lidar; sparse-point proxy; Exp. 5 weights
highres_depth (Faro)	1920 × 1440, ~2.7–4 fps	depth source gt and reference mesh
lowres_wide.traj	~10 Hz	pose (ARKit VIO), slerp/lerp-interpolated to each frame
lowres_wide_intrinsics	per frame	scan median, scaled to resolution

TABLE I

SCENES. EXTENT = AXIS-ALIGNED EXTENT OF THE REFERENCE MESH (WHAT FARO SAW); FARO FRAMES = NUMBER OF FRAMES EVERY ROW IS EVALUATED ON. TOTAL: 2,352 FRAMES / 577 S OF VIDEO.

video_id	Room (3DOD labels)	Extent (m)	Scan (s)	Faro frames	Role
42444474	open-plan kitchen/dining under sloped roof, glass table/rail, brick wall	9.9 × 5.1 × 4.2	88	236	large + glass (dev. scene)
47333774	standard bedroom (bed, shelf, cabinet, table)	5.1 × 4.9 × 3.3	90	375	standard #1
47115299	large living room (4 tables, 2 sofas, TV)	8.1 × 6.1 × 2.5	91	324	large
42897743	bathroom (bathtub, sink, toilet)	8.0 × 3.4 × 3.6	114	448	mirror / tiles
47429736	bedroom + closets (bed, 5 cabinets, 2 tables)	6.6 × 6.4 × 2.7	98	504	standard #2
45261556	kitchen (10 cabinets, stove, sink)	7.1 × 6.1 × 2.4	98	465	glossy #2

Part	Value	Reason
fusion resolution	256 × 192	LiDAR resolution; every source resampled to it
voxel / sdf_trunc	4 cm / 12 cm	Exp. 2 shows 4 cm is the sweet spot
depth range	0.1–5.0 m	cuts near sensor noise and far Faro noise
post-processing	drop clusters < 1000 triangles	no decimation
frame stride	1 (every Faro frame)	except Exp. 3
confidence weights	off	except Exp. 5
device	Apple M-series (MPS)	reported times are on this machine

Aligner	Fitted on	Interpretation
oracle_frame	that frame’s GT depth, every frame	model ceiling (upper bound)
per_scene	GT depth of 10 frames spread over the scan , stacked, fitted once; one (S, T) for all frames	upper bound of a one-off calibration
sparse_points	200 LiDAR pixels/frame at confidence 2 (VIO proxy), per frame; 2 rounds of 10% trimming; < 20 points ⇒ reuse previous (s, t)	upper bound of a VIO-based system

furniture placement); normal consistency = $\text{mean } |n_{pred} \cdot n_{ref}|$ over nearest pairs.

b) *2D (diagnostic)*: Per frame, depth *after* alignment vs. Faro depth on pixels where both lie in 0.1–5 m: RMSE, AbsRel, δ_1 (fraction with $\max(p/g, g/p) < 1.25$), averaged over frames.

c) *Time*: $\text{time_total_s} = \text{data loading} + \text{inference} + \text{alignment} + \text{fusion} + \text{extraction}$ on one machine; inference is reported separately in §VII.

G. Experiments

Every run is one resolved config.yaml plus metrics.json under experiments/results/<exp>/<scene>_<run>/ (all committed). Tables in §V are generated by roomscan report without manual editing; numbers are mean ± std over the six scenes, with per-scene tables in per_scene.md. Everything reproduces with make reference, make sweep-exp1 ... sweep-exp6, make report after downloading the data.

V. RESULTS

All numbers are copied from experiments/results/<exp>/table.md (mean ± std over six scenes) and per_scene.md, generated by roomscan report; the macros for the mono_sparse row are generated by scripts/paper_numbers.py. Every number covers 6 scenes / 2,352 Faro frames unless stated; units are cm; F is the F-score at $\tau = 5$ cm.

A. Exp. 1: depth source (main table)

What the table says directly:

- 1) **The pipeline loses ~2 cm; LiDAR adds ~1 cm.** The gap lidar-gt is 0.98 ± 0.25 cm (0.66–1.33), the same in every room including the bathroom; $F@5$ cm 0.93 vs 0.97. The rows differ clearly only at $\tau = 2$ cm (0.54 vs 0.86): iPad LiDAR is wrong at the 2–4 cm level, not 5 cm.
- 2) **Monocular depth with the right scale on every frame (oracle) is 2.29 ± 1.11 cm behind LiDAR (0.9–3.8).** Bedroom 47333774 (3.8) and the bathroom (3.4) are

Exp.	Variable	Values	Fixed
1 depth source	depth.source × depth.aligner	gt / lidar / mono+oracle / mono+sparse / mono+per_scene / metric+identity	DA-v2 L, 4 cm, stride 1
2 voxel size	fusion.voxel_size	2 / 4 / 8 cm (sdf_trunc = 3×)	gt
3 frame stride	dataset.frame_stride	1 / 5 / 10 / 20 × {gt, mono+oracle, mono+per_scene}	DA-v2 L, 4 cm
4 model size	depth.model	DA-v2 L / DA-v2 S / MiDaS small	oracle_frame, 4 cm
5 LiDAR confidence	lidar_min_confidence, confidence_weights	all / mask _{≥1} / mask=2 / weights [0,1,2] / [1,2,4]	lidar, 4 cm
6 fine-tune teacher	depth.model	pretrained / ft-Faro / ft-LiDAR / ft-LiDAR (24 scenes)	identity, 4 cm

TABLE II
SAME PIPELINE, 4 CM VOXELS, EVERY FARO FRAME; ONLY DEPTH.SOURCE / DEPTH.ALIGNER CHANGE. SPARSE = PER-FRAME FIT ON 200 LiDAR-SAMPLED POINTS (VIO PROXY; UPPER BOUND OF REAL VIO).

Depth source	Aligner	Chamfer	Acc.	Compl.	F@2	F@5	F@10	AbsRel (2D)	Time/scan (s)
Faro laser (gt)	identity	2.1 ± 0.5	1.3	2.8	0.86	0.97	0.99	0	1.6
ARKit LiDAR (lidar)	identity	3.0 ± 0.6	2.2	3.9	0.54	0.93	0.98	0.018	1.7
DA-v2 L (mono)	oracle_frame	5.3 ± 1.3	5.9	4.7	0.46	0.79	0.90	0.032	366
DA-v2 L (mono)	sparse_points	6.0 ± 1.7	6.4	5.5	0.38	0.74	0.88	0.036	360
DA-v2 L (mono)	per_scene	22.9 ± 2.1	24.2	21.6	0.08	0.23	0.41	0.237	351
DA-v2 Metric-Indoor	identity	54.3 ± 10.6	70.3	38.3	0.01	0.03	0.06	0.382	352

TABLE III
CHAMFER (CM) PER SCENE; COLUMN ORDER AS IN TABLE I. 42444474 IS THE DEVELOPMENT SCENE.

Run	42444474 kitchen/glass	47333774 bedroom	47115299 living	42897743 bathroom	47429736 bedroom+closets	45261556 kitchen	mean
gt	1.8	1.2	1.9	2.8	2.4	2.3	2.1
lidar	2.8	2.4	2.9	4.1	3.1	3.0	3.0
mono + oracle	3.8	6.2	4.8	7.5	5.2	4.5	5.3
mono + sparse	4.5	6.1	5.5	9.1	5.9	4.6	6.0
mono + per_scene	24.1	26.0	19.9	23.2	21.4	23.1	22.9
mono metric	48.5	61.6	41.5	48.9	54.1	71.0	54.3

farthest; the development kitchen 42444474 (0.9) and kitchen 45261556 (1.6) closest—more room-dependent than LiDAR’s std suggests (§VI-C).

- 3) **Two hundred sparse metric points per frame recover the oracle.** sparse_points lands at 6.0 cm, 0.6 cm from the oracle and 3.8× better than one-off calibration, with the same per-scene ordering as the oracle (Table III). The remaining gap to LiDAR (2.9 cm) is shape, not scale.
- 4) **One calibration per scene (per_scene) is 4.5±1.1× worse than the oracle (3.1–6.4×), and—importantly—the absolute values cluster at 20–26 cm in every room** (std 2.1 cm, lower than the oracle’s): not one room failing, but a property of affine-invariant models with one scale per scene (§VI-A).
- 5) **The metric model is worst (54.3 cm, F 0.03) and most variable (42–71 cm); accuracy 70.3 cm vs completeness 38.3 cm: the surfaces are all there but in the wrong place (§VI-B).** Kitchen 45261556, where mono+oracle

is second best (4.5), is where the metric model is worst (71.0)—right shape, wrong scale. Fine-tuning this same model on ARKitScenes removes most of that error (§V-F).

B. Exp. 2: voxel size (GT depth)

TABLE IV
DEPTH.SOURCE: GT, SDF_TRUNC = 3× VOXEL.

Voxel	Chamfer	Acc.	Compl.	F@2	F@5	F@10	Fusion (s)
2 cm	1.18 ± 0.19	1.05	1.31	0.92	0.99	1.00	2.5
4 cm	2.06 ± 0.54	1.29	2.83	0.86	0.97	0.99	1.5
8 cm	2.76 ± 0.85	1.74	3.79	0.76	0.93	0.97	1.3

Going from 2 to 4 cm costs 0.9 cm Chamfer, mostly completeness (1.3→2.8: coarser corners and edges), not accuracy (1.05→1.29). From 4 to 8 cm costs another 0.7 cm and F@5 cm drops 0.97→0.93—at 8 cm discretisation exceeds LiDAR’s own error. Fusion takes 1–3 s per scan at every

size (CPU, Open3D), so voxel size does not drive cost; inference does (§VII). **4 cm is used throughout:** its discretisation (2.1 cm) is smaller than the depth-source effects being measured (LiDAR +1, mono +4, per_scene +20) and the mesh stays small enough for a phone.

C. Exp. 3: frame stride—how many frames per second to capture

Faro provides frames at ~ 4.1 fps (2,352 frames / 577 s); strides 1/5/10/20 therefore correspond to $\approx 4 / 0.8 / 0.4 / 0.2$ fps (391 / 78 / 40 / 20 frames per scan on average). Three rows per stride separate the causes: GT control (coverage only), mono+oracle (+ model shape), mono+per_scene (+ scale).

- **Control:** GT accuracy is constant at 1.2–1.3 cm at every stride; the growing Chamfer (2→17 cm) is pure completeness. On this dataset stride measures **coverage**: a room swept once develops holes below 1 fps.
- **mono+oracle tracks the control plus a constant ~ 4 cm**, and its accuracy *improves* with stride (7.1→5.0): fewer frames means fewer overlapping surfaces from different views; 2D AbsRel is 0.040 at every stride, confirming the model did not change.
- **mono+per_scene is flat at ~ 23 cm up to stride 10:** at stride 5 it saves $5\times$ the time (355→72 s) with identical Chamfer (22.7), because the error is already dominated by scale. At stride 20 per_scene fits on 10 of 20 frames (nearly a half-scene oracle), so that cell should not be over-interpreted.
- **For the business:** with good depth (LiDAR) capture ≥ 1 fps for a complete room ($F@5\text{ cm} > 0.9$); with mono+per_scene, 0.8 fps gives the same result as 4 fps, $5\times$ faster—quality is capped by scale, not by frame count.

At stride 20 every row has the same holes (areas the camera passed quickly): holes are a property of the scan, not of the depth source.

D. Exp. 4: model size (oracle scale)

L→S: $13\times$ smaller, $7\times$ faster, +2.8 cm (F 0.74→0.64); every scene moves the same way, the bathroom most (10.7→15.7). S→MiDaS: similar parameter count, a further $4.5\times$ faster, but $1.6\times$ the Chamfer and half the F —a matter of model generation (DA-v2 was trained with 62M unlabelled images), not size. At oracle scale DA-v2 S (9.2 cm) is still 6 cm behind LiDAR (3.0). Model size is not the main bottleneck: Table II shows scale (oracle→per_scene, +16 cm) costs $6\times$ more than model size (L→S, +2.8 cm).

E. Exp. 5: does LiDAR confidence help?

ARKit ships a $\{0, 1, 2\}$ confidence map with every LiDAR frame. Masking or weighting by it is the cheapest possible improvement to the “iPhone Pro today” row. Nothing moves: dropping confidence-0 pixels or weighting by tier changes Chamfer by ≤ 0.03 cm; keeping only confidence-2 pixels improves accuracy slightly (2.23→2.08) but loses completeness (3.85→5.23, worst in the closet-heavy bedroom

47429736: 3.1→5.8). The 1 cm LiDAR—laser gap is not a low-confidence-pixel problem; it is the sensor’s resolution and its 2–4 cm noise everywhere.

F. Exp. 6: fine-tuning with a LiDAR teacher

Exp. 1 leaves one question open: the metric row fails not because monocular depth cannot carry scale, but because a model trained on synthetic indoor data (Hypersim) has never seen an iPad camera. We therefore fine-tuned the same DA-v2 Metric-Indoor checkpoint on ARKitScenes with a frozen DINOv2 backbone (DPT neck + head only, SiLog loss), varying *only the teacher*: Faro highres_depth (R1, 10 scenes), the iPad’s own LiDAR lowres_depth (R2, the same 10 scenes), and LiDAR on 24 scenes (R3, the 10 + 14 scenes with no laser scan at all). Training, validation and checkpoint selection used scenes disjoint from the six test scenes at the visit_id level; at test time every row below is RGB-only with aligner: identity.

- 1) **Fine-tuning turns the worst row into a usable one.** Chamfer drops 54.3 cm→16.6 cm ($3.3\times$) and AbsRel 0.382→0.136, with no alignment and no sensor at test time. $F@5\text{ cm}$ rises from 0.03 to 0.31—still far from the aligned rows, i.e. the surfaces are now roughly where they belong but not within 5 cm.
- 2) **The laser teacher is not what makes it work.** R2 (iPad LiDAR as the label) is $1.13\times$ R1 in Chamfer, identical in AbsRel (0.136 vs 0.136) and *better* in δ_1 (0.848 vs 0.780). Per scene (Table IX) R1 wins four and R2 two, with a spread larger than the difference between their means: with six test scenes the honest claim is that replacing the laser scanner with a consumer depth sensor costs no measurable accuracy, not that the two are equal.
- 3) **Fourteen extra laser-free scenes do not help measurably** (R3: 15.7 cm vs R2’s 16.6 cm). R3 was the best of the three on the validation fold (AbsRel 0.133 vs 0.142) and is not the best here—three validation scenes are too few to rank checkpoints this close, which we report rather than resolve with the test set.
- 4) **Alignment still beats fine-tuning.** sparse_points (6.0 cm) is 11.2 cm better than the best fine-tune. The two are complementary, not alternatives: fine-tuning fixes the scale a model predicts, alignment fixes the scale of the frame it is looking at.

G. Numbers carried into §6–7

VI. DISCUSSION

Material: per-frame scale analysis on 6 scenes (analyze_scale_drift.py, stride 5, 471 frames), the failure-frame study of scene 42444474, and Tables II–III.

A. Why per_scene loses to the oracle by $3.8\times$: the model’s scale follows what is in the frame, not time

An affine-invariant model promises disparity $\approx a \cdot (1/\text{depth}) + b$ with (a, b) depending on the image. If (a, b) were constant over a scan, one calibration (per_scene) would equal the oracle. We measured how constant it is by

TABLE V
CHAMFER / ACCURACY / COMPLETENESS (CM) AND F@5 CM VS. STRIDE.

Stride ($\approx$ fps)	Frames	GT: Ch / acc / comp	F	mono oracle: Ch / acc / comp	F	mono per_scene: Ch / acc / comp	F	mono time (s)
1 (4)	391	2.1 / 1.3 / 2.8	0.97	6.5 / 7.1 / 5.8	0.74	22.7 / 23.8 / 21.7	0.22	355
5 (0.8)	78	4.4 / 1.2 / 7.6	0.92	8.2 / 5.6 / 10.9	0.72	22.7 / 20.3 / 25.2	0.23	72
10 (0.4)	40	8.9 / 1.2 / 16.6	0.83	12.7 / 5.0 / 20.3	0.65	25.0 / 18.9 / 31.0	0.20	36
20 (0.2)	20	16.8 / 1.3 / 32.3	0.68	20.9 / 5.1 / 36.7	0.52	34.5 / 21.4 / 47.6	0.14	19

TABLE VI
ORACLE_FRAME FOR EVERY MODEL, 4 CM, EVERY FRAME—PURE “SHAPE,” EXCLUDING THE SCALE PROBLEM OF §5.1.

Model	Params	Chamfer	F@5	AbsRel	δ_1	ms/frame (MPS)	s/scan
DA-v2 Large	335M	6.45 $\pm$ 2.42	0.74	0.040	0.979	871	355
DA-v2 Small	25M	9.22 $\pm$ 3.48	0.64	0.054	0.964	121	56
MiDaS v2.1 small	21M	14.95 $\pm$ 1.87	0.35	0.110	0.873	27	18

TABLE VII
LiDAR ROW WITH CONFIDENCE MASKING / INTEGER TSDF WEIGHTS (4 CM, EVERY FRAME).

Variant	Chamfer	Acc.	Compl.	F@5
all pixels, weight 1 (Exp. 1)	3.04 ± 0.57	2.23	3.85	0.93
mask confidence ≥ 1	3.07	2.17	3.98	0.93
mask confidence = 2	3.66	2.08	5.23	0.93
weights [0, 1, 2]	3.07	2.17	3.98	0.93
weights [1, 2, 4]	3.04	2.23	3.85	0.93

refitting (s, t) on every fifth frame (Fig. 3, Table X). Three consequences:

- 1) **No number of fit frames helps per_scene:** the (S, T) fitted on 10 spread frames already sits at the median of the oracle values in every scene (e.g. 47333774: $S = 0.00098$ vs. oracle median ≈ 0.0010). The problem is that the median does not apply to most frames, not that it is estimated badly.
- 2) **A $6\times$ per-frame error becomes a $3.8\times$ 3D error** not because TSDF averages it out but because the error concentrates in a minority of frames: frames with per_scene AbsRel > 1 (placed $\geq 2\times$ wrong) are downward shots of table-tops or floor at 0.5–0.8 m and frame-filling blank walls or cabinets. These frames carry no size cue, so the model guesses disparity at “room scale”; multiplied by the room’s (S, T) , a table at 0.5 m lands at 1.5–2 m as a phantom surface in the middle of the room—the red blob in every per_scene panel of Fig. 2.
- 3) **The effect is general, not a property of the development scene:** 47429736 (bedroom+closets) has close-ups in 53/101 frames yet per_scene AbsRel 0.197, the same as a scene with 3/93 (45261556: 0.169); and the five unseen scenes give per_scene Chamfer 19.9–26.2 cm, bracketing 42444474 (23.9).

System implication. The assumption “one scale per scene” is false for this model family on real room video. Two ways remain: (a) *per-frame* metric data—3D feature points from VIO, which ARKit/ARCore deliver with the pose anyway (sparse_points); or (b) drop or down-weight cue-less

TABLE VIII
EXP. 6: SAME PIPELINE, SAME SIX SCENES, RGB-ONLY, NO ALIGNMENT. TEACHER = THE DEPTH USED AS THE TRAINING LABEL; NONE OF THESE ROWS USES A DEPTH SENSOR AT TEST TIME.

Model	Teacher	Chamfer	F@5	AbsRel	δ_1
Metric-Indoor (pretrained)	—	54.3 ± 10.6	0.03	0.382	0.277
+ fine-tune (R1)	Faro	14.6 $\pm$ 4.0	0.31	0.136	0.780
+ fine-tune (R2)	ARKit LiDAR	16.6 ± 4.7	0.23	0.136	0.848
+ fine-tune (R3)	LiDAR, 24 scenes	15.7 ± 4.0	0.24	0.134	0.812

TABLE IX
EXP. 6 CHAMFER (CM) PER SCENE; COLUMN ORDER AS IN TABLE III.

Teacher	4244	4733	4711	4289	4742	4526	mean
none (pretrained)	48.5	61.6	41.5	48.9	54.1	71.0	54.3
Faro (R1)	10.8	13.8	17.6	21.1	12.4	11.9	14.6
LiDAR (R2)	13.9	19.0	12.8	12.5	16.6	24.6	16.6
LiDAR $\times$ 24 (R3)	12.8	16.7	17.3	12.2	12.4	22.5	15.7

frames before fusion—cheaper, but removing close-ups still leaves $4\times$ the oracle, so it can only supplement (a). Exp. 1 confirms (a): 200 points per frame bring the row to 6.0 cm, 0.6 cm from the oracle, with the same per-scene ordering. The per-frame fit needs no history and tolerates frames without points by carrying the previous (s, t) , exactly what a tracker outage looks like.

B. Why the “metric” model gives 54.3 cm: right on average, wrong per frame

DA-v2 Metric-Indoor should output metres directly. Per-frame analysis over the six scenes: In 3D the same surface from several frames is placed at several distances and TSDF averages them into a thick slab: accuracy 70.3 cm but completeness 38.3 cm (Table II)—everything is there, in the wrong place. The worst scene (kitchen 45261556, 71.0 cm) is among the model’s best shapes under the oracle (4.5 cm), confirming a pure scale effect. The worst frames in every scene look alike: a single flat, uniformly coloured surface filling the frame (white cabinets, tiles) with nothing distant to compare against—the model’s prior pushes flat surfaces away. **Implication:** a metric model does not remove calibration; it reduces it from two parameters per frame (s, t) to **one** (s) , which is a real advantage: a handful of VIO points suffice to fit one scalar stably, whereas (s, t) needs points spread over the depth range. The other way to get that scalar is to put it in the weights: fine-tuning the same checkpoint on 3594–19 896 real iPad frames moves the row to 14.6 cm–15.7 cm with no

Question	Number (6 scenes)
pipeline loss at 4 cm	2.1 $\pm$ 0.5 cm
iPad LiDAR vs laser	+1.0 $\pm$ 0.3 cm, F@5 cm 0.93 (confidence weighting: no change)
mono ceiling (scale known per frame)	+2.29 $\pm$ 1.11 cm from LiDAR, F@5 cm 0.79
mono with 200 sparse points/frame	6.0 cm, F@5 cm 0.74 (0.6 cm from oracle)
price of “one scale per scene”	4.5 $\times$ oracle $\rightarrow$ 22.9 cm, F@5 cm 0.23
uncalibrated metric model	54.3 cm, F@5 cm 0.03
fine-tuned on ARKitScenes, RGB-only	14.6 cm (Faro teacher) vs 16.6 cm (LiDAR teacher), F@5 cm 0.31 / 0.23
voxel 2 $\rightarrow$ 4 $\rightarrow$ 8 cm	1.2 $\rightarrow$ 2.1 $\rightarrow$ 2.8 cm
fps needed (LiDAR/GT)	$\geq$ 0.8 fps for F@5 cm $\geq$ 0.9; 0.4 fps leaves 0.83
L $\rightarrow$ S $\rightarrow$ MiDaS	6.5 $\rightarrow$ 9.2 $\rightarrow$ 15.0 cm at 871 $\rightarrow$ 121 $\rightarrow$ 27 ms/frame

run-time fit at all (§V-F), which is the deployable answer for a device that has neither LiDAR nor reliable VIO points.

C. Which rooms are hard, and for whom

Ranking scenes by Chamfer per row (Table III): Faro/LiDAR go from bedroom 47333774 (easiest) to the bathroom (1.2–2.8 / 2.4–4.1); mono+oracle from kitchen 45261556 to the bathroom (4.6–10.7); mono+per_scene from living room 47115299 to bedroom 47333774 (19.9–26.2).

- **The bathroom is hard for every source** (LiDAR +1.3, oracle +6.6 from LiDAR): the mirror yields phantom depth for LiDAR and model alike, glossy tiles reflect; it is the only scene where the oracle fit produced $s \leq 0$ on some frames (a frame-filling mirror). If the target market is bathroom renovation, this paper’s averages are optimistic.
- **Standard bedroom 47333774 is easiest for LiDAR but second-hardest for mono** (oracle 8.1): a small room scanned close to walls and bed leaves most frames at 1–2 m of blank wall with no cue. Rooms with “lots of stuff” (kitchen 45261556: 10 cabinets; living room: 4 tables, 2 sofas) are easy for mono—the model likes objects of known size.
- The mono difficulty order does not match LiDAR’s $\Rightarrow$ **“hard scenes” must be assessed per source**; LiDAR experience does not predict where mono fails.

D. Visible only in 3D: mono disagrees with itself across views

Exp. 3 shows what 2D metrics cannot: reducing frames from 391 to 20 (stride 20) **improves mono+oracle accuracy from 7.1 to 5.1 cm** while 2D AbsRel stays at 0.040 and GT stays at 1.3. Even with every frame perfectly scaled, the shape of the same surface from different views **disagrees** at the $\sim$ 2 cm level, and fusing more frames thickens the surface rather than sharpening it—the multi-view consistency LiDAR has and a single-image model lacks. This is the part of the

oracle $\rightarrow$ LiDAR gap (2.29 cm) that scale alignment cannot fix, and the reason systems like SimpleRecon use multi-image cost volumes (§2.2). The sparse row inherits it: its remaining gap to LiDAR (2.9 cm) is this shape term.

E. Conclusions for system design

VII. BUSINESS IMPLICATIONS

Numbers from §V; times are Apple Silicon / MPS and serve only for ratios (§VIII).

A. The one number a decision-maker needs: “how crooked is the room” per use case

Table XI translates Table II into product language, requiring that a use case have $\geq 90\%$ of the surface within τ of the truth (recall@ τ).

- **iPad Pro today is enough for “furniture placement” but not for “renovation measurement”**: at 2 cm LiDAR reaches only half, and even Faro through this pipeline reaches 0.84 (the 4 cm voxel eats it; §5.2 shows 2 cm voxels reach 0.92). A product promising centimetre accuracy needs finer voxels plus multiple LiDAR passes, or a laser. Confidence weighting does not move this (Exp. 5).
- **Devices without LiDAR serve no use case** with one-off calibration (per_scene 22.9 cm) or a raw metric model (54.3 cm). Fine-tuning that metric model on data a phone can collect itself buys the “rough overview” tier without any run-time calibration (16.6 cm, §V-F) but not the “furniture placement” tier.
- **With per-frame sparse points the same device reaches 6.0 cm, recall@5 cm 0.77**—the “furniture placement” tier at a level close to iPad Pro, and the mono ceiling (oracle 5.3 cm) is only 2.29 cm behind LiDAR. This is the gap to invest in, not the model.

B. Where to invest: scale first, model second

The order the data dictate: **(1) solve per-frame scale $\rightarrow$ (2) ship DA-v2 S on device by default $\rightarrow$ (3) consider L or a compressed model once (1) is stable**. Investing in the model before scale returns < 3 cm on a 22 cm base—invisible in the product.

C. Cost per scan

Per room ($\sim$ 95 s of video $\approx$ 391 frames at 4 fps; MPS, unoptimised PyTorch): At 4 fps DA-v2 L takes 3.6 $\times$ the video length—not real time even on a laptop; DA-v2 S is at 0.5 $\times$ (processable right after capture). Exp. 3 says per_scene quality needs only 0.8 fps (same Chamfer), bringing L to 68 s and S to 9 s per room; with per-frame scale, ≥ 0.8 –4 fps is needed for coverage (F@5 cm 0.72 at 0.8 fps vs 0.74 at 4 fps—0.8 fps is still the sweet spot). Fusion and extraction are a rounding error ($< 3\%$ of L’s time); TSDF is not what to optimise. Data: at 0.8 fps a room is $\sim$ 80 images at 640×480 ($\approx$ 10–20 MB JPEG) plus poses and sparse points—a few seconds to upload, so “capture on device, process on a server with L” is feasible without waiting for on-device models. The roomscan_web package (§III) is that architecture in its minimal form: upload a capture, poll a job, fetch the mesh.

TABLE X
DA-V2 LARGE, PER-SCENE VALUES ACROSS THE SIX SCENES.

	Range over scenes	Meaning
s p95 / p5 within one scene	2.6–4.9 ×	the central 90% of frames need scales 3–5× apart even the middle half swings $\pm 25\%$ not cumulative drift: no direction close-up frames need a different s : view-dependent
s p75 / p25	1.45–1.72	
corr(s , time)	−0.40 ... + 0.28 (sign flips)	
corr(s , frame’s median depth)	−0.27 ... − 0.71 (negative in every scene)	
AbsRel oracle $\rightarrow$ per_scene	0.040 $\rightarrow$ 0.241 (6 ×	in 2D, before fusion, one scale costs 6× close-ups hurt but are not the only cause a disparity shift is needed in every scene
per_scene without close-ups (<1 m)	0.154 (still 4× oracle)	
scale-only per frame, depth space	0.298 $\approx$ per_scene	
	Range over scenes	Meaning
median(GT / pred) per scene	0.69–0.96	the model says 4–31% too far on average (per-scene bias)
GT / pred within a scene	e.g. 0.37–1.22, 0.58–1.80	frames in one room are wrong from −60% to +80%
oracle t in inverse space	−0.08 ... + 0.04 (≈ 0)	no shift: the shape is right in the affine sense
AbsRel raw $\rightarrow$ scale-only/frame $\rightarrow$ oracle (s, t)	0.328 $\rightarrow$ 0.057 $\rightarrow$ 0.046	one scalar per frame recovers 96% of the gap
AbsRel with one scale per scene	0.174	like per_scene for the relative model: the median is not enough
Assumed beforehand	What the data say	
mono’s problem is coarse shape	shape is 2.29 cm behind LiDAR; scale is 18 cm behind	
one calibration per room suffices	no: 22.9 cm in every room; scale is view-dependent	
a metric model removes the scale problem	no: but it reduces it to one parameter per frame	
a metric model cannot be fixed without a laser scanner	fine-tuning on a phone’s own LiDAR is as good a teacher (§V-F)	
a few sparse metric points cannot fix a dense map	they can: 200 points/frame $\rightarrow$ within 0.6 cm of the oracle	
a bigger model means a more accurate room	+2.8 cm S $\rightarrow$ L vs. +16 cm from scale	
more frames mean more accuracy	more frames mean more coverage; mono accuracy improves with fewer frames	
LiDAR confidence maps are free accuracy	no measurable gain from masking or weighting (Exp. 5)	
LiDAR’s hard scenes are mono’s hard scenes	no (blank bedroom: hard for mono, easy for LiDAR)	

TABLE XI
RECALL@ τ AVERAGED OVER 6 SCENES. $\checkmark \geq 0.9$, ~ 0.75 –0.9, $\times < 0.75$.

Use case	τ	Faro	iPad LiDAR	mono + oracle	mono + sparse	mono + per_scene	mono ft (LiDAR)	mono metric
room overview / virtual walk-through	10 cm	$\checkmark$ 0.98	$\checkmark$ 0.97	$\checkmark$ 0.93	$\checkmark$ 0.91	$\times$ 0.47	$\times$ 0.49	$\times$ 0.10
furniture placement / space planning	5 cm	$\checkmark$ 0.96	$\checkmark$ 0.91	$\sim$ 0.82	$\sim$ 0.77	$\times$ 0.27	$\times$ 0.26	$\times$ 0.04
floor-area sale / renovation (walls, trim)	2 cm	$\sim$ 0.84	$\times$ 0.52	$\times$ 0.47	$\times$ 0.39	$\times$ 0.09	$\times$ 0.09	$\times$ 0.01

D. Positioning

What can be said from these data: “*The same system gives 3 cm on a LiDAR device and about 6.0 cm on a device without one, provided the tracker’s sparse points are used per frame—what separates the two numbers is measurable engineering, not model research.*” What **cannot** be said: “no LiDAR, same accuracy”—the data say the opposite in every scene.

E. Risks the numbers do not cover

VIO pose on low-end Android may be worse than ARKit’s, adding error to every row equally (not measured); regions Faro did not see (behind cabinets, high ceilings) are outside the numbers, so a product must handle “the part not captured” itself; the bathroom is the worst scene for every source (LiDAR

4.1, oracle 10.7)—if the target market is bathroom/kitchen renovation, more such scenes must be collected; and the sparse-point row uses LiDAR-sampled points, cleaner than real VIO features (§VIII).

VIII. LIMITATIONS AND FUTURE WORK

A. Limitations of the measurement

(a) **The reference is not an independent ground truth.** It is fused from Faro depth with the same pipeline (1 cm voxels) and uses the **same poses** as every row, so (i) the gt row is non-zero and measures our own discretisation, (ii) VIO pose drift is removed from every number—a real phone system incurs additional pose error this paper does not measure, and (iii) areas Faro did not see are masked from every source

TABLE XII
WHAT EATS MONO’S ERROR, SIX SCENES.

Error source	Size (Chamfer)	Remedy	Cost
one scale per scene → per frame	−18 cm (22.9 → 5.3)	per-frame metric points from VIO (free from ARCore/ARKit)	one aligner (<code>sparse_points</code>); measured: 22.9 → 6.0
teach the model the scale instead	−38 cm (54.3 → 16.6)	fine-tune on scenes captured with a phone’s own LiDAR	one training run (~1 h on an RTX 3070); no run-time cost
model S → L	−2.8 cm (9.2 → 6.5)	13× larger model, 7× slower	on-device / cloud compute
voxel 4 → 2 cm	−0.9 cm	~4× larger mesh	memory / bandwidth
MiDaS → DA-v2 S	−5.7 cm	newer model, same size	zero
LiDAR confidence weighting	±0.03 cm	—	(not worth it)

Stage	LiDAR	DA-v2 L	DA-v2 S	MiDaS s
depth per frame	0 (sensor)	871 ms	121 ms	27 ms
depth per scan @ 4 fps	0	341 s	47 s	11 s
depth per scan @ 0.8 fps (stride 5)	0	68 s	9 s	2 s
align + fusion + extract	1.6 s	~9 s	~6 s	~4 s

	RoomPlan / Polycam (LiDAR)	this work on a LiDAR device	this work without LiDAR, one-off calibration	this work without LiDAR + VIO points
market measured accuracy	iPhone/iPad Pro not disclosed	same 3.0 cm, F@5 cm 0.93	Android + regular iPhone 22.9 cm, F@5 cm 0.23 (16.6 cm fine-tuned, no calibration)	same 6.0 cm, F@5 cm 0.74 (VIO-proxy upper bound)
use case served	(claims floor plan)	furniture placement	not even overview	furniture placement (nearly)
needs from device	LiDAR	LiDAR + VIO	camera + VIO	camera + VIO + feature points

equally, so “completeness” means “complete relative to what Faro saw,” not the whole room.

(b) **Evaluated at ~3 fps only.** Faro depth exists only for the frames Apple kept, so every source is evaluated on that set although LiDAR runs at 60 fps and RGB at 30 fps. What a real pipeline gains or loses from denser frames (smoothing vs. accumulated error) is not measured; Exp. 3 answers coverage, and compute-vs-accuracy only partly.

(c) **Fusion resolution** 256×192 , chosen to be fair to LiDAR, downsamples the monocular model (which predicts at 640×480) before fusion; DA-v2’s sharp-edge advantage does not appear in the 3D numbers. “Mono at its best” would require fusing at higher resolution and accepting a disadvantaged LiDAR row.

(d) **Six rooms, one dataset, one device.** Every scan is an iPad Pro in homes Apple selected; offices, empty rooms and low light are absent, and 42444474 was used during development, so its numbers are in-sample—per-scene tables show whether the other five confirm.

(e) **The fine-tuned rows are selected on three validation scenes.** Exp. 6 keeps the checkpoint with the lowest AbsRel on the 3 validation rooms; that ranking ($R3 > R2 > R1$) does not reproduce on the six test rooms ($R1 \geq R3 \geq R2$ in Chamfer). Three rooms are too few to separate checkpoints that differ by ~0.01 AbsRel, so the Exp. 6 ordering between teachers should be read as “no measurable difference”, and a larger

validation set is the first thing to add. The six test rooms were never used for training, validation or checkpoint choice.

(f) **One fine-tuning recipe, one architecture.** Only DA-v2 Metric-Indoor Large with a frozen encoder was fine-tuned; whether unfreezing the encoder, a different loss or another architecture would widen or close the Faro-vs-LiDAR teacher gap is untested, as is training on more than 24 rooms.

(g) **Times are Apple Silicon (MPS) with unoptimised PyTorch,** not representative of on-device inference (CoreML / NNAPI); use them for *ratios* between models, not absolute values.

B. Limitations of the system

(a) **per_scene still calibrates on GT:** it is the upper bound of one-off calibration; a real system using a measured wall or camera height would do worse. §VI-A shows even this bound is far from the oracle because scale swings per view—the assumption, not the calibration method, is the problem.

(b) **sparse_points is evaluated on a proxy.** ARKitScenes ships no ARKit feature points, so the aligner sees 200 pixels sampled from the LiDAR frame at high confidence. Real VIO points are triangulated, noisier, and cluster on texture; the row is therefore an *upper bound* of a VIO-based system. The loader exposes a noise knob (`sparse_noise`) and point count for sensitivity studies; an evaluation on

real ARKit/ARCore point clouds needs our own captures (dataio/custom.py reads them).

(c) **No outlier rejection before fusion:** every posed frame is integrated equally. §VI-A shows a minority of frames (close-ups, blank walls) create most of `per_scene`’s error; frame weighting is what Exp. 5’s mechanism provides, applied here only to LiDAR confidence so the main tables measure the depth source alone.

(d) **Poses come from the dataset:** there is no tracking in the system; an MVP uses ARKit/ARCore poses of similar quality to the trajectories used here, possibly worse on low-end Android.

(e) **No GPU fusion:** fusion is <2s per scan on CPU, so it is not a bottleneck; a GPU backend fits behind the same TSDFusion interface if higher resolutions are pursued.

C. Future work (ordered by impact on the MVP decision)

- 1) **Real VIO points:** capture rooms with the app (ARKit `rawFeaturePoints` / ARCore point cloud) plus a reference (tape measurements or a rented laser scanner) and rerun the `sparse_points` row on real, noisy, textured points; sweep `sparse_noise` and point count on ARKitScenes meanwhile.
- 2) **Metric model + scale-only per frame:** §VI-B suggests one parameter per frame suffices; test whether one VIO point per frame is enough.
- 3) **Frame selection / weighting** for mono: drop or down-weight frames with median depth <1 m or low disparity variance before fusion, using the weight channel of ADR-012.
- 4) **Android capture (ARCore depth source):** `depth_sources/arcore.py` has its registry slot; data must be collected with a reference.
- 5) **Compressed models:** Exp. 4 provides the L / S / MiDaS baseline; a distilled or quantised model plugs in as `models/<x>.py` and reruns Exp. 4.
- 6) **Fusion above 256×192 for mono,** to measure what downsampling costs DA-v2.
- 7) **Semantic layer** (wall / floor / furniture labels on the mesh) for usable-area computation; sits after post-processing and does not affect the geometry numbers.

IX. CONCLUSION

We built a classical room-reconstruction pipeline (per-frame depth → scale alignment → TSDF → mesh) in which **the depth source is the single variable switched from config**, and used it to measure how many centimetres Faro laser, iPad LiDAR and monocular models deviate **under identical conditions** on six real rooms with a laser-scanner reference. The numbers say:

- 1) **The pipeline loses 2.1 ± 0.5 cm at 4 cm voxels and iPad LiDAR adds only 1.0 ± 0.3 cm** (Chamfer 3.0 cm, F@5 cm 0.93), uniformly across room types; LiDAR’s confidence map does not change this. A LiDAR device today serves 5 cm-level use cases but not 2 cm-level ones.

- 2) **Monocular depth with the right scale on every frame is 2.29 ± 1.11 cm behind LiDAR** (5.3 cm, F@5 cm 0.79)—the size of the “Android vs. iPhone Pro” gap *if* scale is solved.
- 3) **Monocular depth’s problem is not shape but a scale that swings per view:** one calibration per scene gives 22.9 ± 2.1 cm ($4.5\times$ the oracle) in every room, and an uncalibrated “metric” model 54.3 cm; per-frame analysis shows the model’s scale varies severalfold within one scan, uncorrelated with time but correlated with what is in the frame.
- 4) **Two hundred sparse metric points per frame—what a phone’s tracker provides—recover the oracle:** 6.0 cm, F@5 cm 0.74, within 0.6 cm of the ceiling, as a LiDAR-proxy upper bound. The remedy is per-frame metric data, not a better model or a finer calibration.
- 5) **The scale can also be put into the weights:** fine-tuning the metric model’s head on ARKitScenes brings the uncalibrated row from 54.3 cm to 14.6 cm, and **a teacher costs the same whether it is a laser scanner or the LiDAR in a phone** (16.6 cm, $1.13\times$; identical AbsRel)—so the training data for the devices without a depth sensor can be collected by the devices that have one.
- 6) **Model size matters $6\times$ less than scale:** DA-v2 Small costs 2.8 cm for a $7\times$ speed-up (121 ms/frame); MiDaS small is $4.5\times$ faster again with $2.3\times$ Large’s error.
- 7) **On ARKitScenes, frame count governs coverage, not accuracy:** GT accuracy is constant at 1.2–1.3 cm at every stride; ≥ 0.8 fps is needed for a complete room (F@5 cm ≥ 0.9).

The systems result: separating `DepthSource` and `ScaleAligner` into interfaces made all six experiments (33 runs $\times$ 6 scenes) pure YAML, with the orchestrator edited once in the whole project (two lines for confidence weights, recorded as a decision). Finding 3 exists because the aligner was measurable in isolation—had scale been hidden inside the model, one would see only “mono is bad” without knowing why; finding 4 was a single new row in the same table. The same code, behind a small upload/queue/view service and a loader for the app’s capture folder, is the product’s backend; the next step is to replace the LiDAR-sampled proxy with real ARKit/ARCore feature points on rooms we capture ourselves.

REFERENCES

- [1] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [2] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, “Depth anything: Unleashing the power of large-scale unlabeled data,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 10 371–10 381.
- [3] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth anything V2,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

- [4] S. F. Bhat, R. Birkel, D. Wofk, P. Wonka, and M. Müller, “ZoeDepth: Zero-shot transfer by combining relative and metric depth,” *arXiv preprint arXiv:2302.12288*, 2023.
- [5] W. Yin, C. Zhang, H. Wang, S. Niklaus, L. Mai, S. Chen, and C. Shen, “Metric3D: Towards zero-shot metric 3D prediction from a single image,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 9043–9053.
- [6] L. Piccinelli, Y.-H. Yang, C. Sakaridis, M. Segu, S. Li, L. Van Gool, and F. Yu, “UniDepth: Universal monocular metric depth estimation,” in *CVPR*, 2024.
- [7] A. Bochkovskii, A. Delaunoy, H. Germain, M. Santos, Y. Zhou, S. R. Richter, and V. Koltun, “Depth pro: Sharp monocular metric depth in less than a second,” in *ICLR*, 2025.
- [8] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Joffe, D. Kurz, A. Schwartz, and E. Shulman, “ARKitScenes: A diverse real-world dataset for 3D indoor scene understanding using mobile RGB-D data,” in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [9] B. Curless and M. Levoy, “A volumetric method for building complex models from range images,” in *Proc. SIGGRAPH*, 1996, pp. 303–312.
- [10] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, “KinectFusion: Real-time dense surface mapping and tracking,” in *Proc. IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011, pp. 127–136.
- [11] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv preprint arXiv:1801.09847*, 2018.
- [12] Z. Murez, T. van As, J. Bartolozzi, A. Sinha, V. Badrinarayanan, and A. Rabinovich, “Atlas: End-to-end 3D scene reconstruction from posed images,” in *Proc. European Conference on Computer Vision (ECCV)*, 2020, pp. 414–431.
- [13] J. Sun, Y. Xie, L. Chen, X. Zhou, and H. Bao, “NeuralRecon: Real-time coherent 3D reconstruction from monocular video,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 598–15 607.
- [14] M. Sayed, J. Gibson, J. Watson, V. Prisacariu, M. Firman, and C. Gdard, “SimpleRecon: 3D reconstruction without 3D convolutions,” in *Proc. European Conference on Computer Vision (ECCV)*, 2022, pp. 1–19.
- [15] K. Tateno, F. Tombari, I. Laina, and N. Navab, “CNN-SLAM: Real-time dense monocular SLAM with learned depth prediction,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6243–6252.
- [16] Z. Yu, S. Peng, M. Niemeyer, T. Sattler, and A. Geiger, “MonoSDF: Exploring monocular geometric cues for neural implicit surface reconstruction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [17] Apple Inc., “RoomPlan – apple developer documentation,” <https://developer.apple.com/documentation/roomplan>, 2022, accessed 2026-09-17.

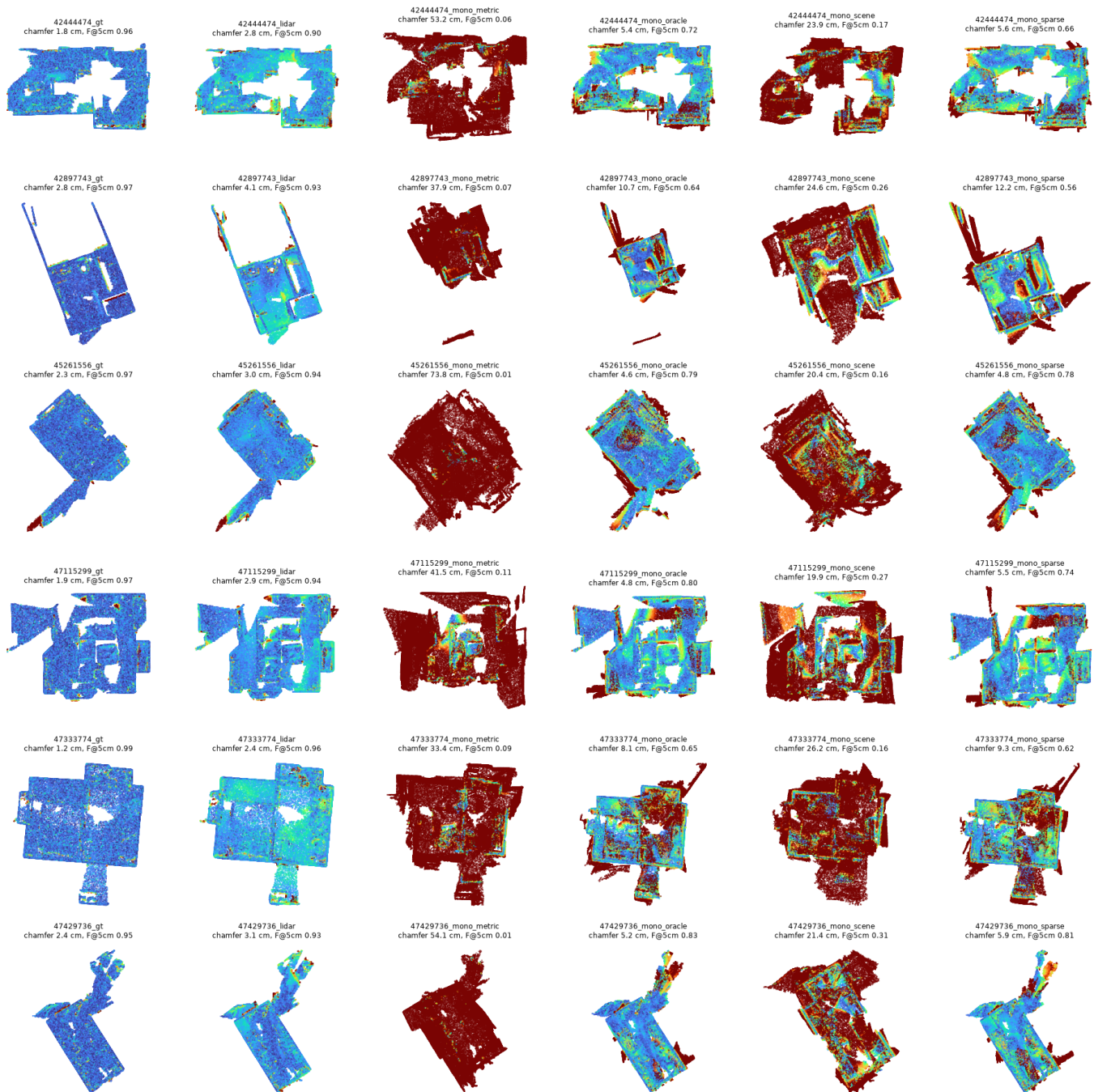


Fig. 2. Exp. 1 top-down error maps (rows = scenes, columns = runs, colour = distance to reference, 0–10 cm). The gt/lidar columns are almost entirely blue (<2 cm); oracle and sparse show the same yellow–red pattern at object edges and far walls; per_scene has a red blob in the middle of every room (phantom surfaces from wrongly scaled frames); metric is red almost everywhere.

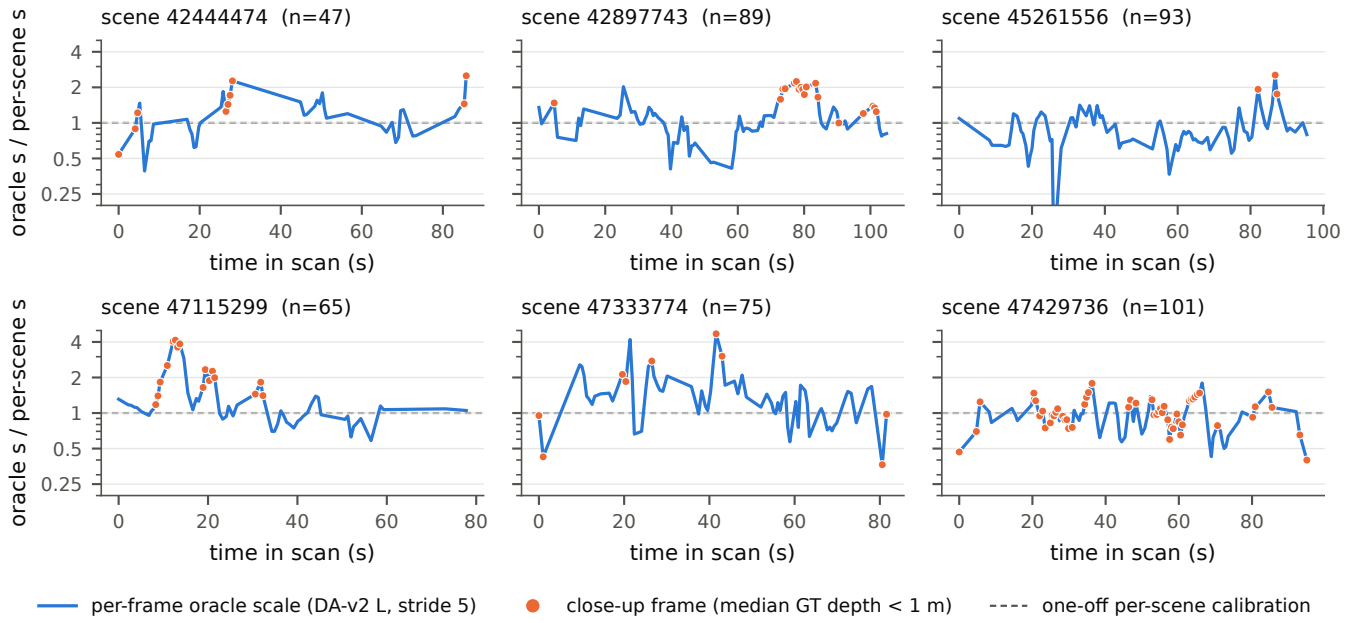


Fig. 3. Per-frame oracle scale divided by the per-scene scale, over time, for DA-v2 Large on all six scenes (log axis; 1.0 = the one-off calibration was right for that frame). Orange dots mark close-up frames (median GT depth < 1 m). The ratio swings by 2–4 $\times$ within every scan with no monotone trend.

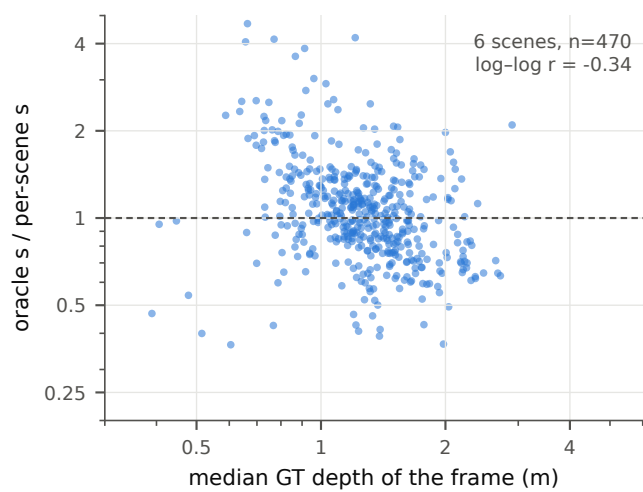


Fig. 4. The same ratio against the frame's median GT depth, all scenes pooled: the scale a frame needs correlates with how close the camera is to what it sees.